



Tutoraggio: Fondamenti di Informatica

A.A. 2025/2026

Lezione #3



Esercizio for (1):

Vediamo come ve la cavate

```
1 #a
2 for x in range(0, 5) :
3     print ("ancora una")
4 #b
5 for x in range(1, 5) :
6     print ("ancora una")
7 #c
8 for x in range(0, 5, 2) :
9     print ("ancora una")
10 #d
11 for x in range(0, 5, 5) :
12     print ("ancora una")
13 #e
14 for x in range(1, 5, 5) :
15     print ("ancora una")
16 #f
17 for x in range(1, 5, -2) :
18     print ("ancora una")
19 #g
20 for x in range(5, 1, -2) :
21     print ("ancora una")
```



Esercizio for (1):

```
1 #a
2 for x in range(0, 5) :
3     print ("ancora una")
4 #b
5 for x in range(1, 5) :
6     print ("ancora una")
7 #c
8 for x in range(0, 5, 2) :
9     print ("ancora una")
10 #d
11 for x in range(0, 5, 5) :
12     print ("ancora una")
13 #e
14 for x in range(1, 5, 5) :
15     print ("ancora una")
16 #f
17 for x in range(1, 5, -2) :
18     print ("ancora una")
19 #g
20 for x in range(5, 1, -2) :
21     print ("ancora una")
```



Esercizio for (1):

```
1 #a
2 for x in range(0, 5) :
3     print ("ancora una")
4 #b
5 for x in range(1, 5) :
6     print ("ancora una")
7 #c
8 for x in range(0, 5, 2) :
9     print ("ancora una")
10 #d
11 for x in range(0, 5, 5) :
12     print ("ancora una")
13 #e
14 for x in range(1, 5, 5) :
15     print ("ancora una")
16 #f
17 for x in range(1, 5, -2) :
18     print ("ancora una")
19 #g
20 for x in range(5, 1, -2) :
21     print ("ancora una")
```

5



range crea una lista di 5
elementi che sono:
[0,1,2,3,4]

Esercizio for (1):

```
1 #a
2 for x in range(0, 5) :
3     print ("ancora una")
4 #b
5 for x in range(1, 5) :
6     print ("ancora una")
7 #c
8 for x in range(0, 5, 2) :
9     print ("ancora una")
10 #d
11 for x in range(0, 5, 5) :
12     print ("ancora una")
13 #e
14 for x in range(1, 5, 5) :
15     print ("ancora una")
16 #f
17 for x in range(1, 5, -2) :
18     print ("ancora una")
19 #g
20 for x in range(5, 1, -2) :
21     print ("ancora una")
```



range crea una lista di 4
elementi che sono:
[1,2,3,4]

Esercizio for (1):

```
1 #a
2 for x in range(0, 5) :
3     print ("ancora una")
4 #b
5 for x in range(1, 5) :
6     print ("ancora una")
7 #c
8 for x in range(0, 5, 2) :
9     print ("ancora una")
10 #d
11 for x in range(0, 5, 5) :
12     print ("ancora una")
13 #e
14 for x in range(1, 5, 5) :
15     print ("ancora una")
16 #f
17 for x in range(1, 5, -2) :
18     print ("ancora una")
19 #g
20 for x in range(5, 1, -2) :
21     print ("ancora una")
```

5

4

3



range crea una lista di 3
elementi che sono:
[0,2,4]

Esercizio for (1):

```
1 #a
2 for x in range(0, 5) :
3     print ("ancora una")
4 #b
5 for x in range(1, 5) :
6     print ("ancora una")
7 #c
8 for x in range(0, 5, 2) :
9     print ("ancora una")
10 #d
11 for x in range(0, 5, 5) :
12     print ("ancora una")
13 #e
14 for x in range(1, 5, 5) :
15     print ("ancora una")
16 #f
17 for x in range(1, 5, -2) :
18     print ("ancora una")
19 #g
20 for x in range(5, 1, -2) :
21     print ("ancora una")
```

5

4

3

1



range crea una lista di **unico**
elemento:
[0]

Esercizio for (1):

```
1 #a
2 for x in range(0, 5) :
3     print ("ancora una")
4 #b
5 for x in range(1, 5) :
6     print ("ancora una")
7 #c
8 for x in range(0, 5, 2) :
9     print ("ancora una")
10 #d
11 for x in range(0, 5, 5) :
12     print ("ancora una")
13 #e
14 for x in range(1, 5, 5) :
15     print ("ancora una")
16 #f
17 for x in range(1, 5, -2) :
18     print ("ancora una")
19 #g
20 for x in range(5, 1, -2) :
21     print ("ancora una")
```

5

4

3

1

1



range crea una lista di **unico**
elemento:
[1]

Esercizio for (1):

```
1 #a
2 for x in range(0, 5) :
3     print ("ancora una")
4 #b
5 for x in range(1, 5) :
6     print ("ancora una")
7 #c
8 for x in range(0, 5, 2) :
9     print ("ancora una")
10 #d
11 for x in range(0, 5, 5) :
12     print ("ancora una")
13 #e
14 for x in range(1, 5, 5) :
15     print ("ancora una")
16 #f
17 for x in range(1, 5, -2) :
18     print ("ancora una")
19 #g
20 for x in range(5, 1, -2) :
21     print ("ancora una")
```

5

4

3

1

1

0



range crea una lista vuota,
perchè i parametri di input
non rispettano le condizione
imposte dalla funzione

Esercizio for (1):

```
1 #a
2 for x in range(0, 5) :
3     print ("ancora una")
4 #b
5 for x in range(1, 5) :
6     print ("ancora una")
7 #c
8 for x in range(0, 5, 2) :
9     print ("ancora una")
10 #d
11 for x in range(0, 5, 5) :
12     print ("ancora una")
13 #e
14 for x in range(1, 5, 5) :
15     print ("ancora una")
16 #f
17 for x in range(1, 5, -2) :
18     print ("ancora una")
19 #g
20 for x in range(5, 1, -2) :
21     print ("ancora una")
```

5

4

3

1

1

0

2

range crea una lista di 2
elementi:
[5, 3]

Esercizio for (3) :

Tre semplici esercizi di uso del comando for (un po' più complessi dei precedenti):

- a) Scrivere una funzione Python stampaPosPari() che riceve come parametro una sequenza contenente 7 numeri interi (p.es. una sequenza come questa: [3, 5, 12, 31, 43, 7, 11]) e visualizza sullo schermo (usando il comando print inserito in un opportuno ciclo for) solo i numeri che si trovano nelle posizioni pari della sequenza (quindi in posizione 0, 2, 4, 6);
- b) come sopra, ma scrivendo questa volta una funzione stampaPosDispari() che visualizza sullo schermo solo i numeri che si trovano nelle posizioni dispari della sequenza;
- c) Domanda: le funzioni che avete scritto per risolvere gli esercizi (a) e (b) sono in grado di operare su sequenze di qualunque lunghezza? (quindi non solo sequenze di lunghezza uguale a 7). In caso positivo, verificate la correttezza, usando come parametro sequenze di lunghezza diversa da 7; in caso negativo, provate a generalizzare le funzioni in modo che lavorino su sequenze di numeri di qualunque lunghezza (quindi non solo su sequenze di 7 numeri).



Esercizio for (3) :

3	5	12	31	43	7	11
---	---	----	----	----	---	----



Esercizio for (3) :

Posizione pari

3	5	12	31	43	7	11
<u>0</u>	1	<u>2</u>	3	<u>4</u>	5	<u>6</u>



Esercizio for (3): Posizione dispari

3	5	12	31	43	7	11
0	<u>1</u>	2	<u>3</u>	4	<u>5</u>	6



Esercizio for (3) :

Posizione pari

```
1 def stampaPosPari_1():
2     vettore=[3,5,12,31,43,7,11]
3     print "vettore[",0,"]=",vettore[0]
4     print "vettore[",2,"]=",vettore[2]
5     print "vettore[",4,"]=",vettore[4]
6     print "vettore[",6,"]=",vettore[6]
7
8 stampaPosPari_1()
```



Esercizio for (3):

Posizione pari

```
1 def stampaPosPari_1():
2     vettore=[3,5,12,31,43,7,11]
3     print "vettore[",0,"]=",vettore[0]
4     print "vettore[",2,"]=",vettore[2]
5     print "vettore[",4,"]=",vettore[4]
6     print "vettore[",6,"]=",vettore[6]
7
8 stampaPosPari_1()
```

```
vettore[ 0 ]= 3
vettore[ 2 ]= 12
vettore[ 4 ]= 43
vettore[ 6 ]= 11
```



Esercizio for (3): Posizione pari

```
1 def stampaPosPari_1():
```

Can we do better?

```
8 stampaPosPari_1()
```

```
for i in range(0, 10, 2):
```



Esercizio for (3): Meno codice

```
1 def stampaPosPari_2():
2     vettore = [3,5,12,31,43,7,11]
3     for i in range(0,7,2):
4         print "vettore[" ,i, "]= ",vettore[i]
5
6 stampaPosPari_2()
```



Esercizio for (3): Meno codice

```
1 def stampaPosPari_2():
```

Can we do better?

```
6 stampaPosPari_2()
```

Esercizio for (3): Più flessibile

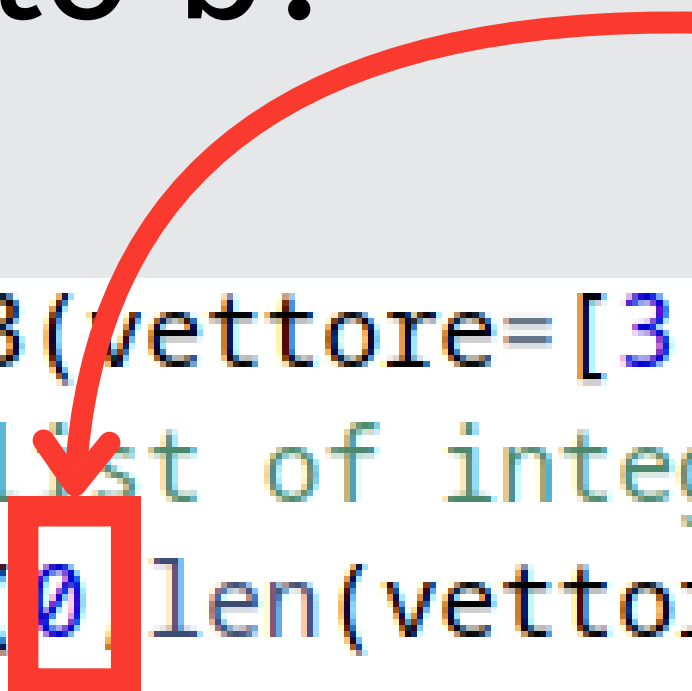
```
1 def stampaPosPari_3(vettore=[3,5,12,31,43,7,11]):  
2     # @Param vettore: list of integer  
3     for i in range(0,len(vettore),2):  
4         print "vettore[" ,i, "]= ",vettore[i]  
5  
6 stampaPosPari_3()
```



Esercizio for (3):

E per il fare il punto b?

```
1 def stampaPosPari_3(vettore=[3,5,12,31,43,7,11]):  
2     # @Param vettore: list of integer  
3     for i in range(0, len(vettore), 2):  
4         print "vettore[" , i, "]=", vettore[i]  
5  
6 stampaPosPari_3()
```



Esercizio for (3): Posizione dispari

```
1 def stampaPosDispari_3(vettore=[3,5,12,31,43,7,11]):  
2     # @Param vettore: list of integer  
3     for i in range(1,len(vettore),2):  
4         print "vettore[" ,i, "]= ",vettore[i]  
5  
6 stampaPosDispari_3()
```

Esercizio for (3) :

Punto C

```
1 def stampaPosDispari(vettore=[3,5,12,31,43,7,11]):
2     # @Param vettore: list of integer
3     for i in range(1,len(vettore),2):
4         print "vettore[",i,"]=",vettore[i]
5
6 def stampaPosPari(vettore=[3,5,12,31,43,7,11]):
7     # @Param vettore: list of integer
8     for i in range(0,len(vettore),2):
9         print "vettore[",i,"]=",vettore[i]
10
11 stampaPosPari()
12 stampaPosDispari()
```

Esercizio for (3):

Punto C

```
1 def stampaPosDispari(vettore=[3,5,12,31,43,7,11]):  
2     # @Param vettore: list of integer
```

Can we do better?

```
9         print "vettore[" ,i, "]= ",vettore[i]  
10  
11 stampaPosPari()  
12 stampaPosDispari()
```


Esercizio for (3):

Punto C (very few code)

```
1 def stampaPos(vettore=[3,5,12,31,43,7,11],dispari=0):  
2     # @Param vettore: list of integer  
3     for i in range(dispari,len(vettore),2):  
4         print "vettore[",i,"]=",vettore[i]
```

```
6 stampaPos()  
7 stampaPos(dispari=1)  
8 v=[3,5,6,7,8,9,0,2,35,34]  
9 stampaPos(vettore=v)  
10 stampaPos(vettore=v,dispari=1)
```

Esercizio 3 - Gestione immagini

A) Scrivete una funzione che, data un'immagine, visualizza (usando il comando print) la media aritmetica delle componenti Red di tutti i suoi pixel che hanno la coordinata $x \leq K$, dove K è un valore intero passato come parametro (per coordinata x si intende la coordinata "larghezza")

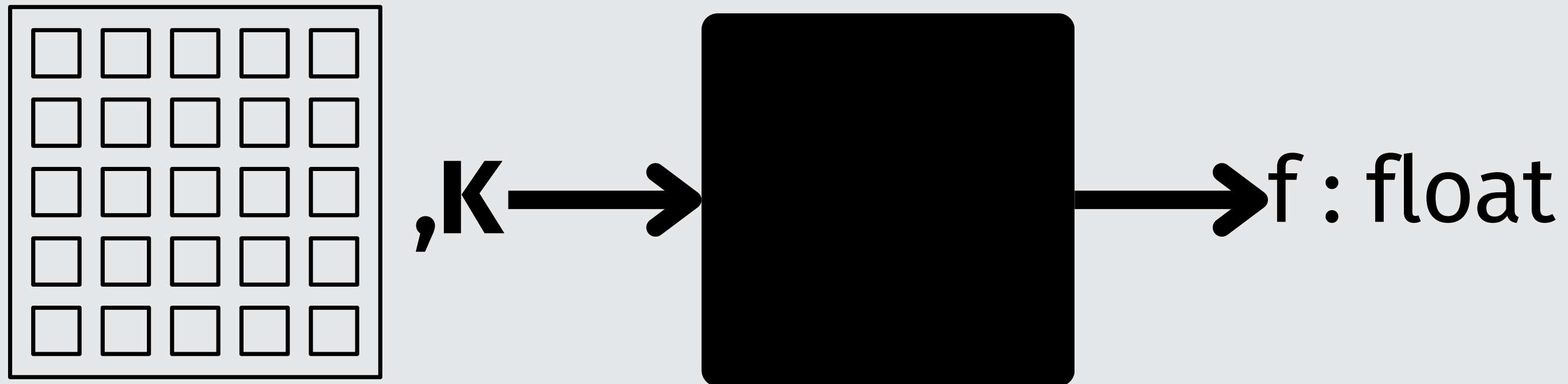
B) Data la funzione `privacy2(pict, xmin, xmax, ymin, ymax)` presente nelle slide già distribuite, provate a modificare il suo codice in modo da evitare a priori la possibilità di errori se la funzione viene chiamata associando ai parametri formali `xmin`, `xmax`, `ymin`, `ymax` valori che stanno fuori dalle dimensioni dell'immagine `pict`.

(Domanda preliminare: data un'immagine `pict`, quali sono gli intervalli di valori ammissibili per `xmin`, `xmax`, `ymin`, `ymax`?)

Suggerimento: nel linguaggio Python, sono predefinite le funzioni `min()`, `max()` ...

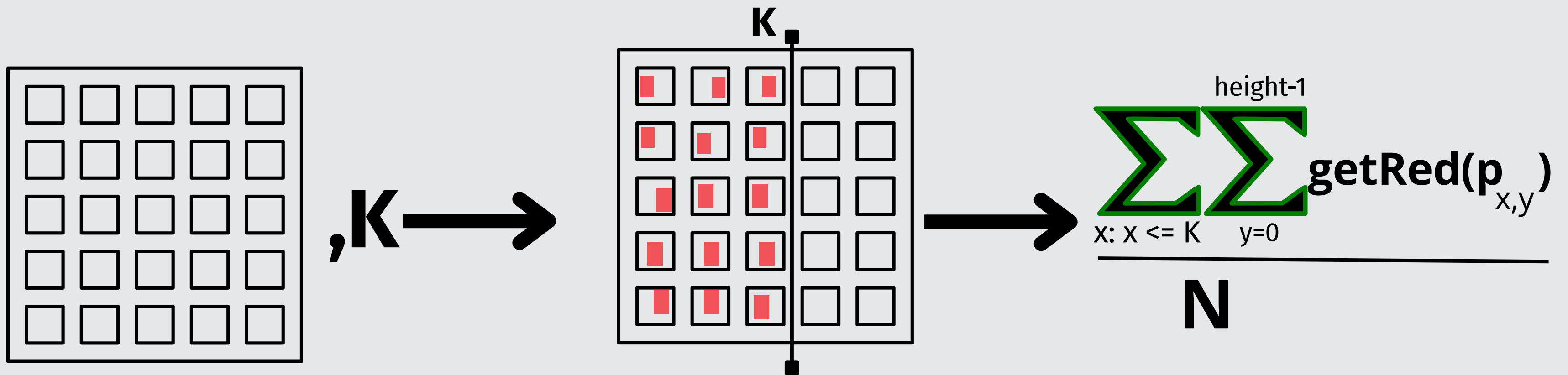
Esercizio 3 - A

Scrivete una funzione che, data un'immagine, visualizza (usando il comando print) la media aritmetica delle componenti Red di tutti i suoi pixel che hanno la coordinata $x \leq \underline{K}$, dove $\underline{K}$ è un valore intero passato come parametro (per coordinata x si intende la coordinata "larghezza")



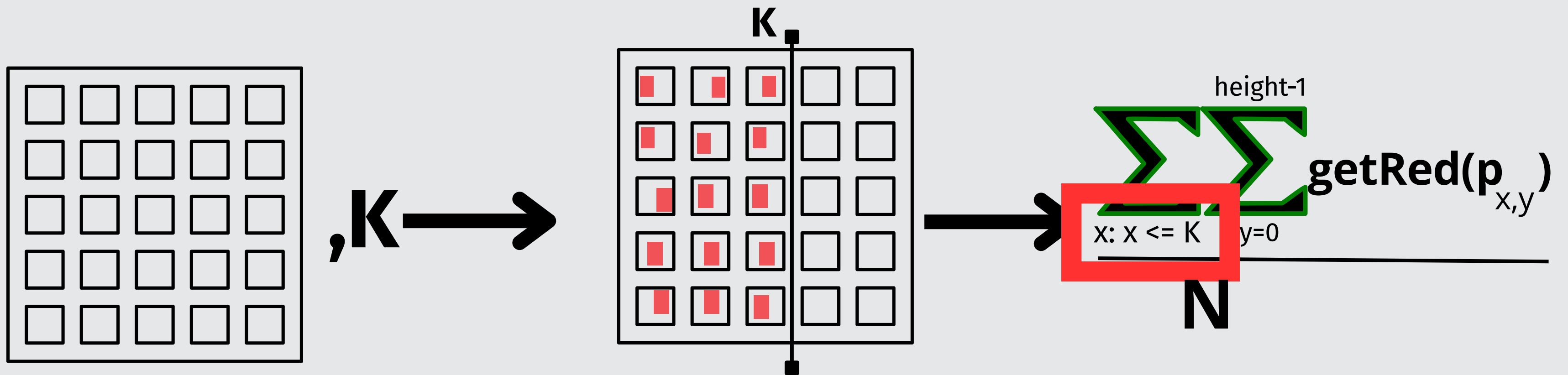
Esercizio 3 - A

Scrivete una funzione che, data un'immagine, visualizza (usando il comando print) la media aritmetica delle componenti Red di tutti i suoi pixel che hanno la coordinata $x \leq K$, dove K è un valore intero passato come parametro (per coordinata x si intende la coordinata "larghezza")



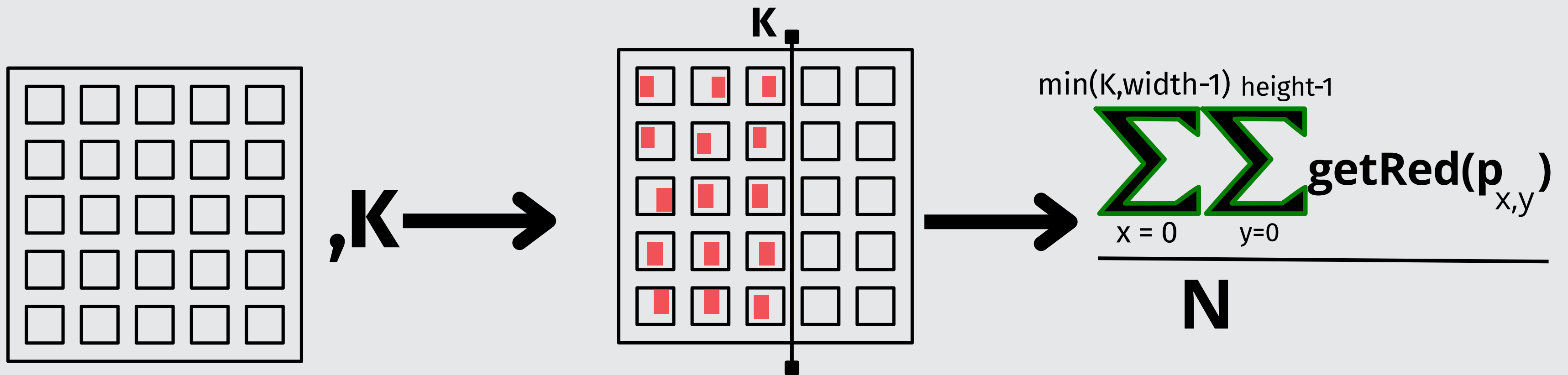
Esercizio 3 - A

Scrivete una funzione che, data un'immagine, visualizza (usando il comando print) la media aritmetica delle componenti Red di tutti i suoi pixel che hanno la coordinata $x \leq K$, dove K è un valore intero passato come parametro (per coordinata x si intende la coordinata "larghezza")



Esercizio 3 - A

Scrivete una funzione che, data un'immagine, visualizza (usando il comando print) la media aritmetica delle componenti Red di tutti i suoi pixel che hanno la coordinata $x \leq \underline{K}$, dove $\underline{K}$ è un valore intero passato come parametro (per coordinata x si intende la coordinata "larghezza")



Esercizio 3 - A

Cosa ci serve?

- 1) Sappiamo gestire l'immagine per matrice?
- 2) Sappiamo quali valore come inizio e fine dei cicli ?
- 3) Sappiamo contare il numero di pixel che utilizziamo ?
- 4) Sappiamo prendere la componente rossa di un pixel ?
- 5) Sappiamo la formula della media ?

Esercizio 3 - A

Cosa ci serve?

1) Sappiamo gestire l'immagine per matrice?

Un ciclo **for** esterno che scorre su una dimensione e d'un ciclo **for** interno che scorra sull'altra dimensione, e l'utilizzo della funzione **getPixel**

2) Sappiamo quali valore come inizio e fine dei cicli ?

3) Sappiamo contare il numero di pixel che utilizziamo ?

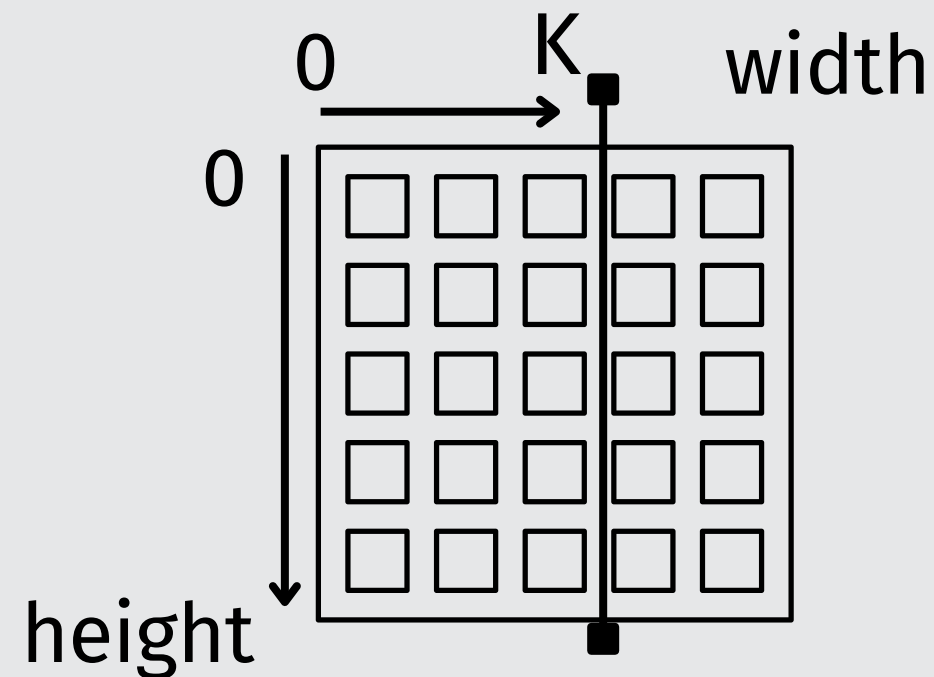
4) Sappiamo prendere la componente rossa di un pixel ?

5) Sappiamo la formula della media ?

Esercizio 3 - A

Cosa ci serve?

- 1) Sappiamo gestire l'immagine per matrice?
- 2) Sappiamo quali valore come inizio e fine dei cicli ?

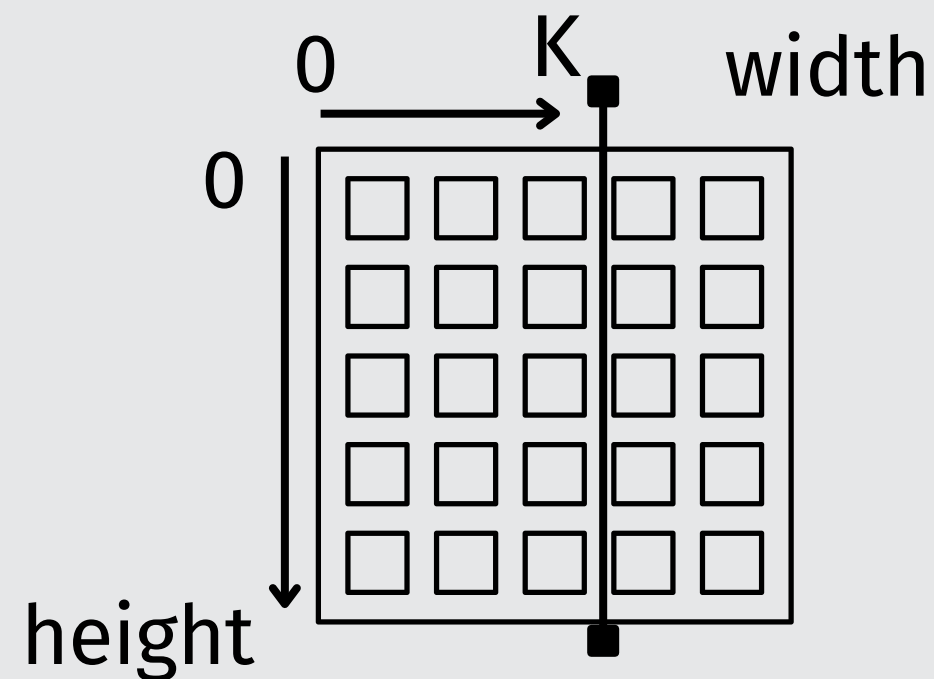


- 3) Sappiamo contare il numero di pixel che utilizziamo ?
- 4) Sappiamo prendere la componente rossa di un pixel ?
- 5) Sappiamo la formula della media ?

Esercizio 3 - A

Cosa ci serve?

- 1) Sappiamo gestire l'immagine per matrice?
- 2) Sappiamo quali valore come inizio e fine dei cicli ?



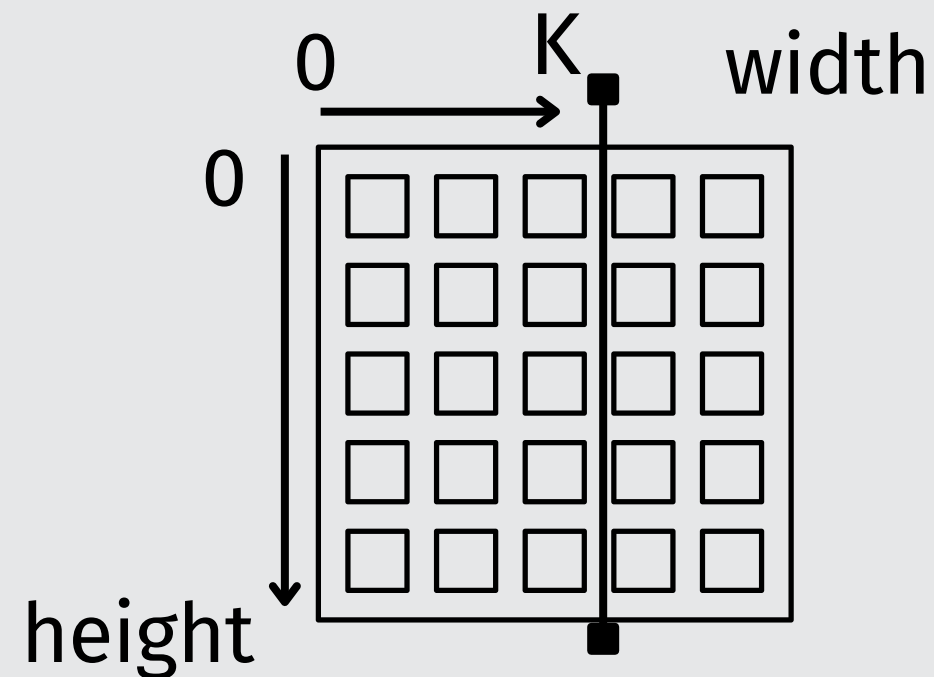
E se K maggiore di width?

- 3) Sappiamo contare il numero di pixel che utilizziamo ?
- 4) Sappiamo prendere la componente rossa di un pixel ?
- 5) Sappiamo la formula della media ?

Esercizio 3 - A

Cosa ci serve?

- 1) Sappiamo gestire l'immagine per matrice?
- 2) Sappiamo quali valore come inizio e fine dei cicli ?



- $0 \leq x \leq \min(K, \text{width})$
- $0 \leq y \leq \text{height}$

- 3) Sappiamo contare il numero di pixel che utilizziamo ?
- 4) Sappiamo prendere la componente rossa di un pixel ?
- 5) Sappiamo la formula della media ?

Esercizio 3 - A

Cosa ci serve?

- 1) Sappiamo gestire l'immagine per matrice?
- 2) Sappiamo quali valore come inizio e fine dei cicli ?
- 3) Sappiamo contare il numero di pixel che utilizziamo ?

Contatore all'interno dei cicli **for** che si incrementa ogni iterazione!

- 4) Sappiamo prendere la componente rossa di un pixel ?
getRed(pixel)

- 5) Sappiamo la formula della media?

$$\text{media aritmetica} = \frac{\sum_{i=0}^n a_i}{n}$$

Esercizio 3 - A

Soluzione

```
1 def esercizio3(pic,K):
2     # @Param pic: Pictures
3     # @Param K: integer
4     width = getWidth(pic)
5     height = getHeight(pic)
6     upperBound = min(K+1,width)
7     tot = 0.0    #variabile accumulatore dei valori delle componente rosse dei pixel
8     n = 0        #variabile che permette di contare i valori che devo sommare
9     for x in range(0,upperBound):
10         for y in range(0,height):
11             p = getPixel(pic,x,y)
12             n = n + 1
13             cRed = getRed(p)
14             tot = tot + cRed
15     media = tot/n
16     print "la media delle componenti Red dei pixel con componente x minore o uguale di ",K," e': ",media
```


Esercizio 3 - A

Soluzione Alternativa

```
1 def esercizio3(pic,K):
2 # @Param pic: Pictures
3 # @Param K: integer
4 width = getWidth(pic)
5 height = getHeight(pic)
6 upperBound = min(K+1,width)
7 tot = 0.0 #variabile accumulatore dei valori delle componente rosse dei pixel
8 n = upperBound * height
9 for x in range(0,upperBound):
10     for y in range(0,height):
11         p = getPixel(pic,x,y)
12         cRed = getRed(p)
13         tot = tot + cRed
14 media = tot/n
15 print "la media delle componenti Red dei pixel con componente x minore o uguale di ",K," e': ",media
```

Esercizio 3 - B

Data la funzione `privacy2(pict, xmin, xmax, ymin, ymax)` presente nelle slide già distribuite, provate a modificare il suo codice in modo da evitare a priori la possibilità di errori se la funzione viene chiamata associando ai parametri formali `xmin`, `xmax`, `ymin`, `ymax` valori che stanno fuori dalle dimensioni dell'immagine `pict`.

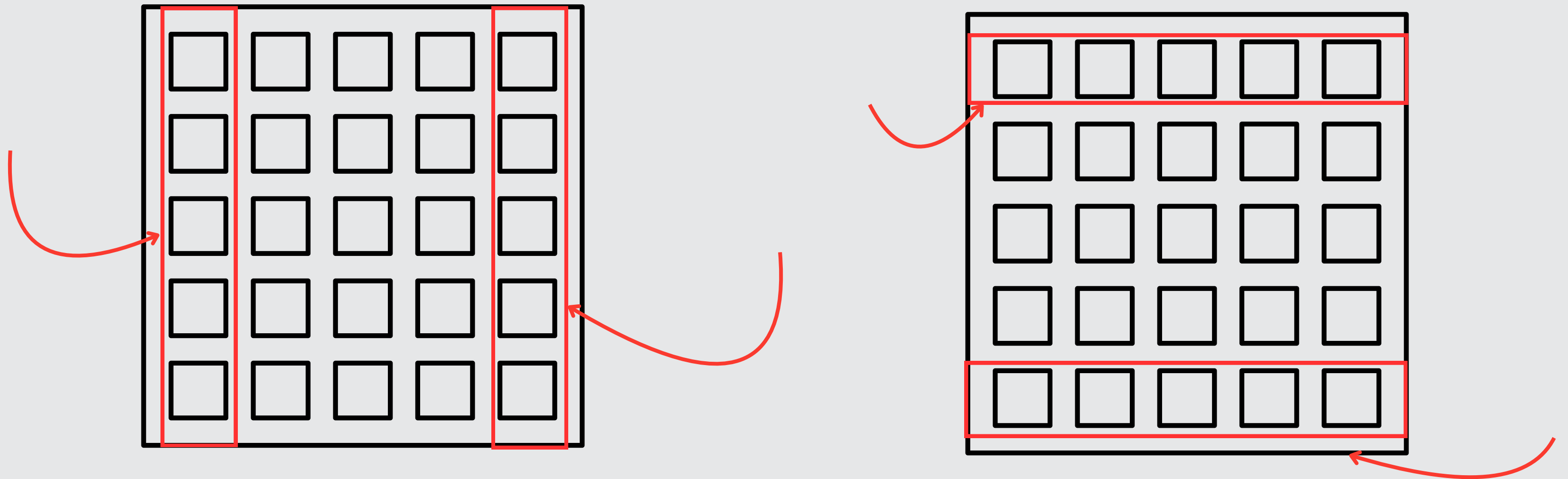
(Domanda preliminare: data un'immagine `pict`, quali sono gli intervalli di valori ammissibili per `xmin`, `xmax`, `ymin`, `ymax`?)

Suggerimento: nel linguaggio Python, sono predefinite le funzioni `min()`, `max()` ...

Esercizio 3 - B

Domanda preliminare

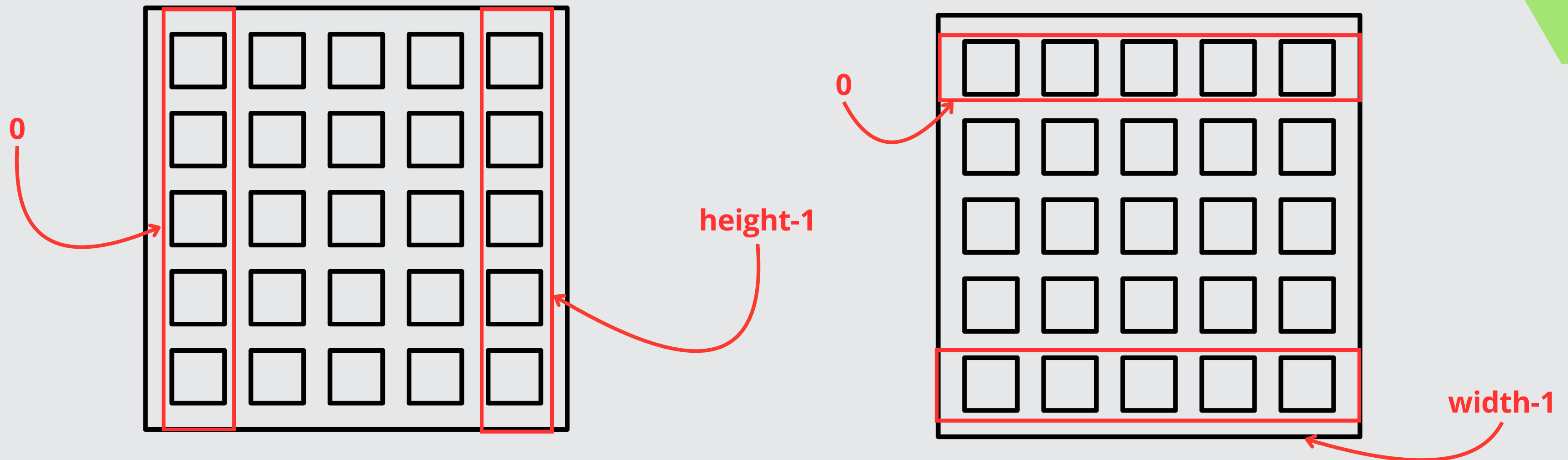
Data un'immagine pict, quali sono gli intervalli di valori ammissibili per x_{min} , x_{max} , y_{min} , y_{max} ?



Esercizio 3 - B

Domanda preliminare

Data un'immagine pict, quali sono gli intervalli di valori ammissibili per xmin, xmax, ymin, ymax?



Esercizio 3 - B

```
1 def privacy2(pic,xmin,xmax,ymin,ymax):
2     # @Param pic: Pictures
3     # @Param xmin: integer
4     # @Param xmax: integer
5     # @Param ymin: integer
6     # @Param ymax: integer
7     for x in range(xmin,xmax):
8         for y in range(ymin,ymax):
9             pix = getPixel(pict,x,y)
10            setColor(pix,black)
```

Esercizio 3 - B

```
1 def privacy2(pict,xmin,xmax,ymin,ymax):
2     # @Param pict: Pictures
3     # @Param xmin: integer
4     # @Param xmax: integer
5     # @Param ymin: integer
6     # @Param ymax: integer
7     height = getHeight(pict)
8     width = getWidth(pict)
9     # questo permette di scambiare i valori in caso di dimensioni invertite
10    xL = min(xmin,xmax)
11    yL= min(ymin,ymax)
12    xU = max(xmin,xmax)
13    yU = max(ymin,ymax)
14    #questo permette di capire bene i limiti su cui devono essere eseguiti i cicli
15    lBoundX = max(0,xL)
16    uBoundX = min(width,xU)
17    lBoundY = max(0,yL)
18    uBoundY = min(height,yU)
19    for x in range(lBoundX,uBoundX):
20        for y in range(lBoundY,uBoundY):
21            pix = getPixel(pict,x,y)
22            setColor(pix,black)
```

Esercizio 3 - B

```
1 def privacy2(pict,xmin,xmax,ymin,ymax):
2     # @Param pict: Pictures
3     # @Param xmin: integer
4     # @Param xmax: integer
5     # @Param ymin: integer
6     # @Param ymax: integer
7     height = getHeight(pict)
8     width = getWidth(pict)
9     # questo permette di scambiare i valori in caso di dimensioni invertite
10    xL = min(xmin,xmax)
11    yL= min(ymin,ymax)
12    xU = max(xmin,xmax)
13    yU = max(ymin,ymax)
14    #questo permette di capire bene i limiti su cui devono essere eseguiti i cicli
15    lBoundX = max(0,xL)
16    uBoundX = min(width,xU)
17    lBoundY = max(0,yL)
18    uBoundY = min(height,yU)
19    for x in range(lBoundX,uBoundX):
20        for y in range(lBoundY,uBoundY):
21            pix = getPixel(pict,x,y)
22            setColor(pix,black)
```

Check extra!

impostazioni
dei limiti

Esercizio :

Uso del comando for in struttura bi-dimensionale (A)

Svolgere gli esercizi proposti nel set di slide "Slide_04_b" – "Problemi per casa" 1, 2, 3) :

- 1) scrivere una funzione che consenta di inserire un rettangolo colorato all'interno di un'immagine (posizione, colore e dimensione del rettangolo forniti come parametri);
- 2) scrivere una funzione che traccia su un'immagine quadrata la diagonale e l'antidiagonale;
- 3) Scrivere una funzione che traccia una "scaletta" all'interno di un'immagine bianca.



Esercizio :

Uso del comando **for in** in struttura bi-dimensionale (A1)

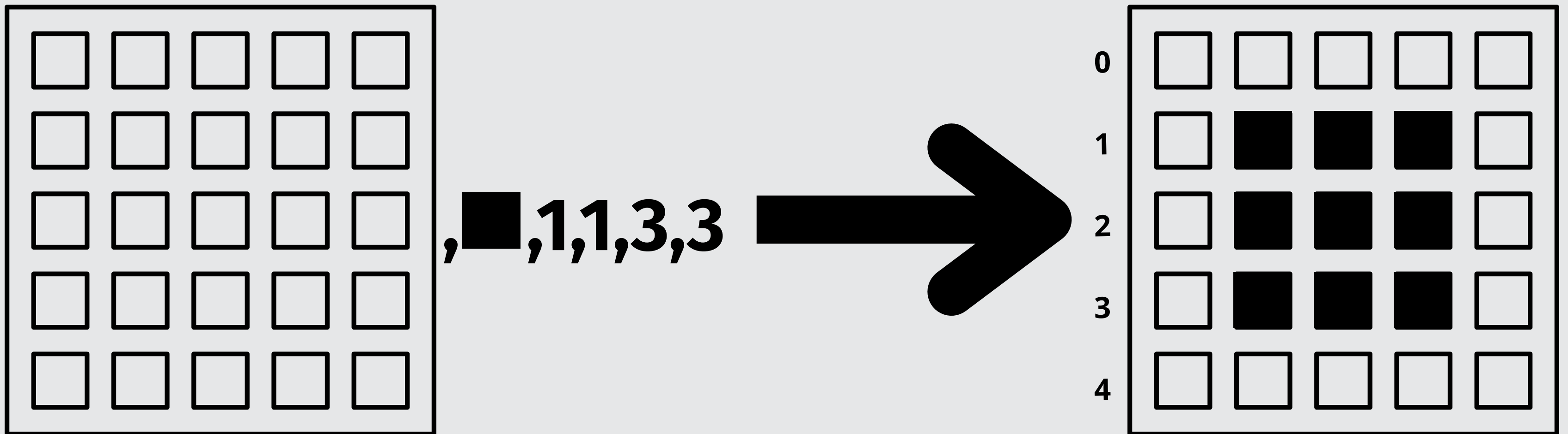
Scrivere una funzione che consenta di inserire un rettangolo colorato all'interno di un'immagine (**posizione**, **colore** e **dimensione** del rettangolo forniti come parametri);

N.B: JES mette a disposizione questa funzione bisogna re-implementarla



Esercizio :

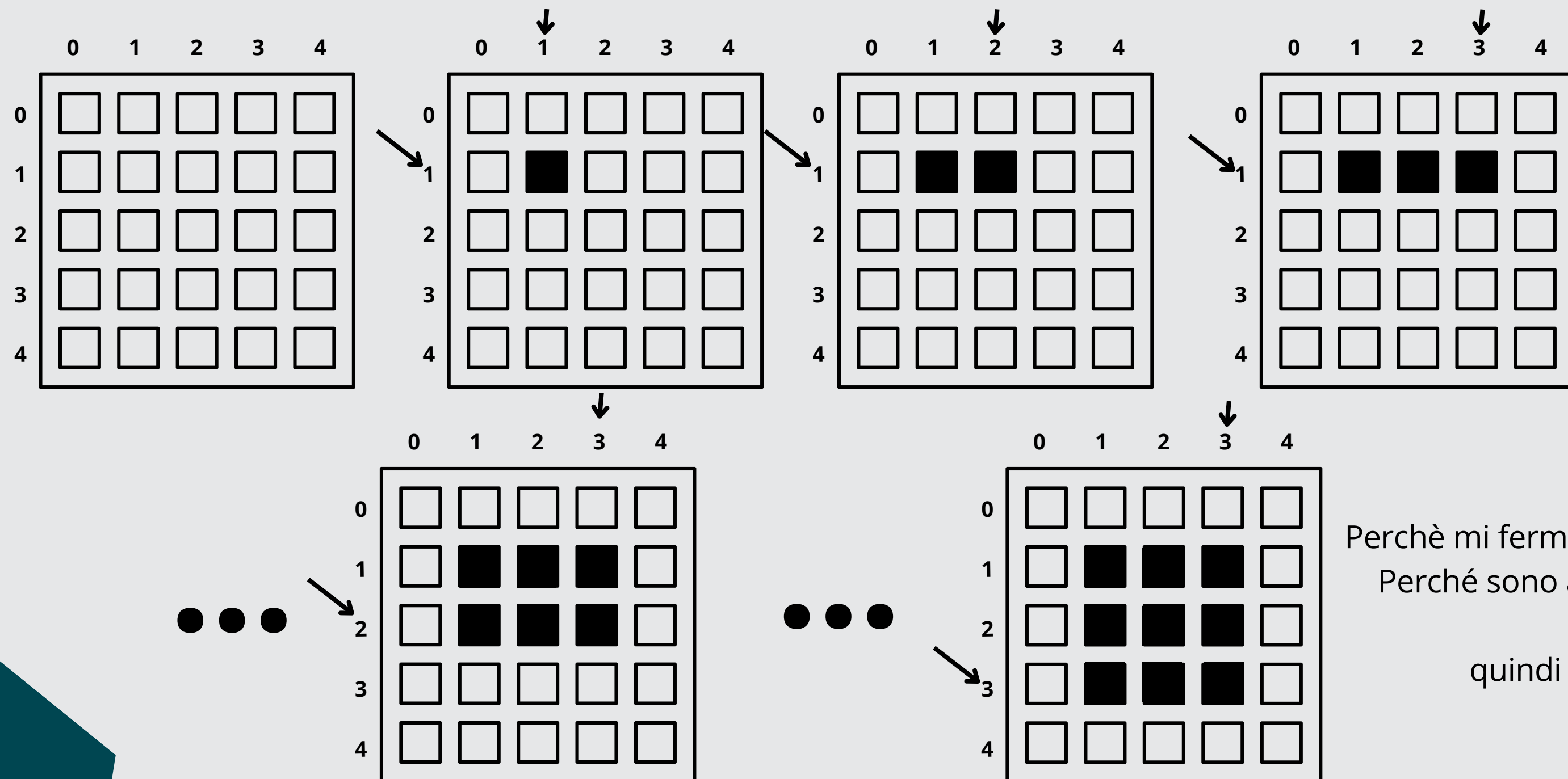
Uso del comando for in struttura bi-dimensionale (A1)



Esercizio :

Uso del comando for in struttura bi-dimensionale (A1)

Supponiamo per semplicità una cosa **assurda**, i parametri che ci passano vanno sempre **bene**



Perché mi fermo con questa riga?
Perché sono arrivato a fare 3 posizioni,
quindi **xStart+width**

Perché mi fermo con vedere le colonne?
Perché sono arrivato a cambiare 3 righe,
quindi **yStart+height**

Esercizio :

Uso del comando for in struttura bi-dimensionale (A1)

Soluzione Facile

```
1 def esercizioA1(pict,col,xStart,yStart,height,width):
2     # @Param pict:Picture
3     # @Param col:Color
4     # @Param xStart:int
5     # @Param yStart:int
6     # @Param height:int
7     # @Param width:int
8     for x in range(xStart,xStart+width):
9         for y in range(yStart,yStart+height):
10             p = getPixel(pict,x,y)
11             setColor(p,col)
```

Esercizio :

Uso del comando for in struttura bi-dimensionale (A1)

Quali controlli bisogna fare?

Sicuramente dobbiamo verificare che:

- $0 < xStart$
- $0 < yStart$
- $xStart + width \leq pictWidth$
- $yStart + height \leq pictHeight$



Esercizio :

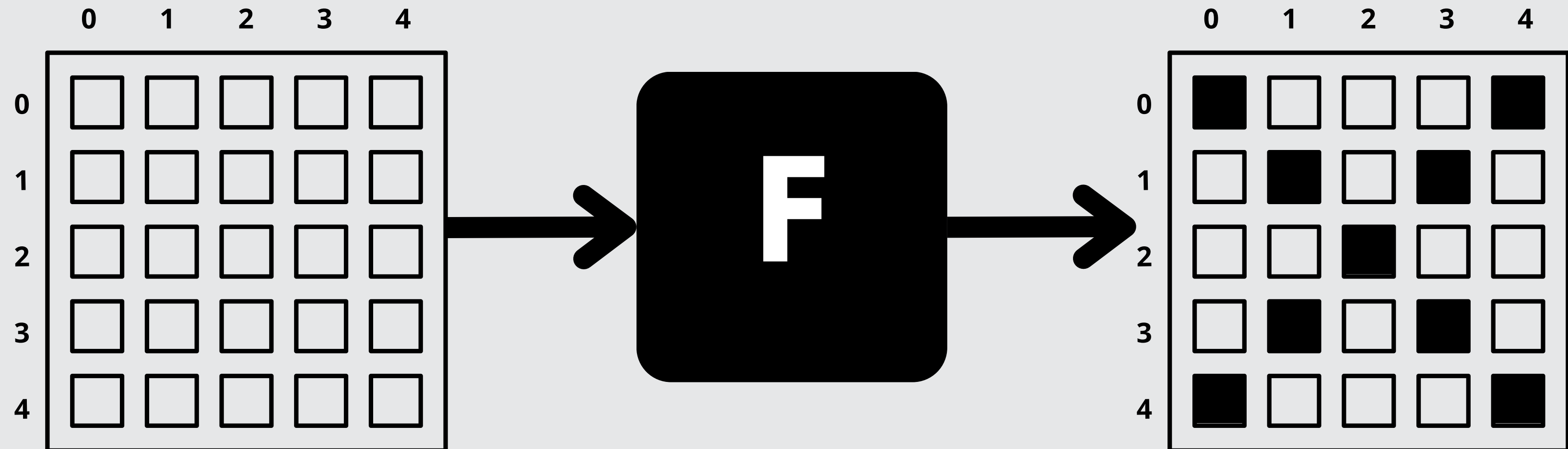
Uso del comando for in struttura bi-dimensionale (A1)

```
1 def esercizioA1(pict,col,xStart,yStart,height,width):
2     # @Param pict:Picture
3     # @Param col:Color
4     # @Param xStart:int
5     # @Param yStart:int
6     # @Param height:int
7     # @Param width:int
8     hPict = getHeight(pict)
9     wPict = getWidth(pict)
10    xStart = max(0,xStart)
11    yStart = max(0,yStart)
12    yUpperBound = min(height+yStart,hPict)
13    xUpperBound = min(width+xStart,wPict)
14    for x in range(xStart,xUpperBound):
15        for y in range(yStart,yUpperBound):
16            p = getPixel(pict,x,y)
17            setColor(p,col)
```

Esercizio :

Uso del comando `for in` in struttura bi-dimensionale (A2)

Scrivere una funzione che traccia su un'immagine quadrata la diagonale e l'antidiagonale;



Esercizio :

Uso del comando for in struttura bi-dimensionale (A2)

Cosa ci serve?

- **Un modo per discriminare i pixel che fanno parte della diagonale principale e quelli per l'anti-diagonale?**
- **Supponiamo di conoscere una formula magica, tipo oracolo, che ci dica dato un pixel se fa parte della diagonale/antidiagonale o meno, come potremmo utilizzare questa informazione per non farlo su tutti i pixel?**



Esercizio :

Uso del comando for in struttura bi-dimensionale (A2)

	0	1	2	3	4
0	■	□	□	□	□
1	□	■	□	□	□
2	□	□	■	□	□
3	□	□	□	■	□
4	□	□	□	□	■

Ogni elemento sulla diagonale di una matrice quadrata ha una caratteristica:

gli indici x e y hanno lo stesso valore.

$(0,0), (1,1), \dots, (42,42), (323,232), \dots$

sono tutti elementi che possono appartenere alla diagonale

Esercizio :

Uso del comando for in struttura bi-dimensionale (A2)

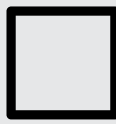
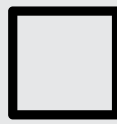
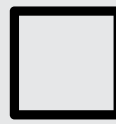
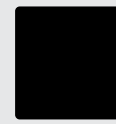
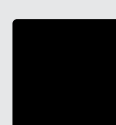
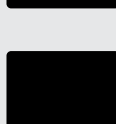
Ogni elemento sull'antidiagonale di una matrice quadrata rispetta la seguente caratteristica:

La somma degli indici x e y è uguale ad una dimensione - 1.

se abbiamo una matrice 3,3:

(0,2),(1,1) e (2,0)

sono tutti elementi che possono appartenere alla diagonale

	0	1	2	3	4
0					
1					
2					
3					
4					

Esercizio :

Uso del comando for in struttura bi-dimensionale (A2)

Soluzione

```
1 def EsercizioA2(pict,col):
2     #@Param pict:Picture
3     #@Param col:color
4     h = getHeight(pict)
5     w = getWidth(pict)
6     if h!=w:
7         return
8     for x in range(0,w):
9         for y in range(0,h):
10             if x == y:
11                 p = getPixel(pict,x,y)
12                 setColor(p,col)
13     for x in range(0,w):
14         for y in range(0,h):
15             if x + y == w - 1:
16                 p = getPixel(pict,x,y)
17                 setColor(p,col)
```


Esercizio :

Uso del comando for in struttura bi-dimensionale (A2)

Soluzione

check se l'immagine è
effettivamente quadrata!

```
1 def EsercizioA2(pict,col):
2     #@Param pict:Picture
3     #@Param col:color
4     h = getHeight(pict)
5     w = getWidth(pict)
6     if h!=w:
7         return
8     for x in range(0,w):
9         for y in range(0,h):
10             if x == y:
11                 p = getPixel(pict,x,y)
12                 setColor(p,col)
13     for x in range(0,w):
14         for y in range(0,h):
15             if x + y == w - 1:
16                 p = getPixel(pict,x,y)
17                 setColor(p,col)
```

Esercizio :

Uso del comando for in struttura bi-dimensionale (A2)

Soluzione

check se l'immagine è
effettivamente quadrata!

Disegno la Diagonale
principale

```
1 def EsercizioA2(pict,col):
2     #@Param pict:Picture
3     #@Param col:color
4     h = getHeight(pict)
5     w = getWidth(pict)
6     if h!=w:
7         return
8     for x in range(0,w):
9         for y in range(0,h):
10             if x == y:
11                 p = getPixel(pict,x,y)
12                 setColor(p,col)
13     for x in range(0,w):
14         for y in range(0,h):
15             if x + y == w - 1:
16                 p = getPixel(pict,x,y)
17                 setColor(p,col)
```

Esercizio :

Uso del comando for in struttura bi-dimensionale (A2)

Soluzione

check se l'immagine è
effettivamente quadrata!

Disegno la Diagonale
principale

Disegno l'antidiagonale

```
1 def EsercizioA2(pict,col):
2     #@Param pict:Picture
3     #@Param col:color
4     h = getHeight(pict)
5     w = getWidth(pict)
6     if h!=w:
7         return
8     for x in range(0,w):
9         for y in range(0,h):
10             if x == y:
11                 p = getPixel(pict,x,y)
12                 setColor(p,col)
13     for x in range(0,w):
14         for y in range(0,h):
15             if x + y == w - 1:
16                 p = getPixel(pict,x,y)
17                 setColor(p,col)
```

Esercizio :

Uso del comando for in struttura bi-dimensionale (A2)

Soluzione

check se l'immagine è

```
1 def EsercizioA2(pict,col):  
2     #@Param pict:Picture  
3     #@Param col:color
```

Can we do better?

Disegno l'antidiagonale

```
11         p = getPixel(pict,x,y)  
12         setColor(p,col)  
13     for x in range(0,w):  
14         for y in range(0,h):  
15             if x + y == w - 1:  
16                 p = getPixel(pict,x,y)  
17                 setColor(p,col)
```

Esercizio :

Uso del comando for in struttura bi-dimensionale (A2)

Soluzione

Uniamo i due cicli in un
unico ciclo!
Stessi **if** ma dentro lo
gli stessi cicli!

```
1 def EsercizioA2(pict,col):
2     #@Param pict:Picture
3     #@Param col:color
4     h = getHeight(pict)
5     w = getWidth(pict)
6     if h!=w:
7         return
8     for x in range(0,w):
9         for y in range(0,h):
10             if x == y:
11                 p = getPixel(pict,x,y)
12                 setColor(p,col)
13             if x + y == w - 1:
14                 p = getPixel(pict,x,y)
15                 setColor(p,col)
```


Esercizio :

Uso del comando for in struttura bi-dimensionale (A2)

Soluzione

```
1 def EsercizioA2(pict,col):  
2     #@Param pict:Picture
```

Can we do better?

gli stessi cicli!

```
10     for y in range(0,n):  
11         if x == y:  
12             p = getPixel(pict,x,y)  
13             setColor(p,col)  
14         if x + y == w - 1:  
15             p = getPixel(pict,x,y)  
16             setColor(p,col)
```

Esercizio :

Uso del comando for in struttura bi-dimensionale (A2)

Soluzione definitiva.

Abbiamo delle condizioni che sono equazioni!

- dato una $x \rightarrow y=x$ per la diagonale principale
- dato una $x \rightarrow y=w-1-x$ per la diagonale principale

```
1 def EsercizioA2(pict,col):
2     #@Param pict:Picture
3     #@Param col:color
4     h = getHeight(pict)
5     w = getWidth(pict)
6     if h!=w:
7         return
8     for x in range(0,w):
9         p = getPixel(pict,x,x)
10        setColor(p,col)
11        p = getPixel(pict,x,w-1-x)
12        setColor(p,col)
13
```

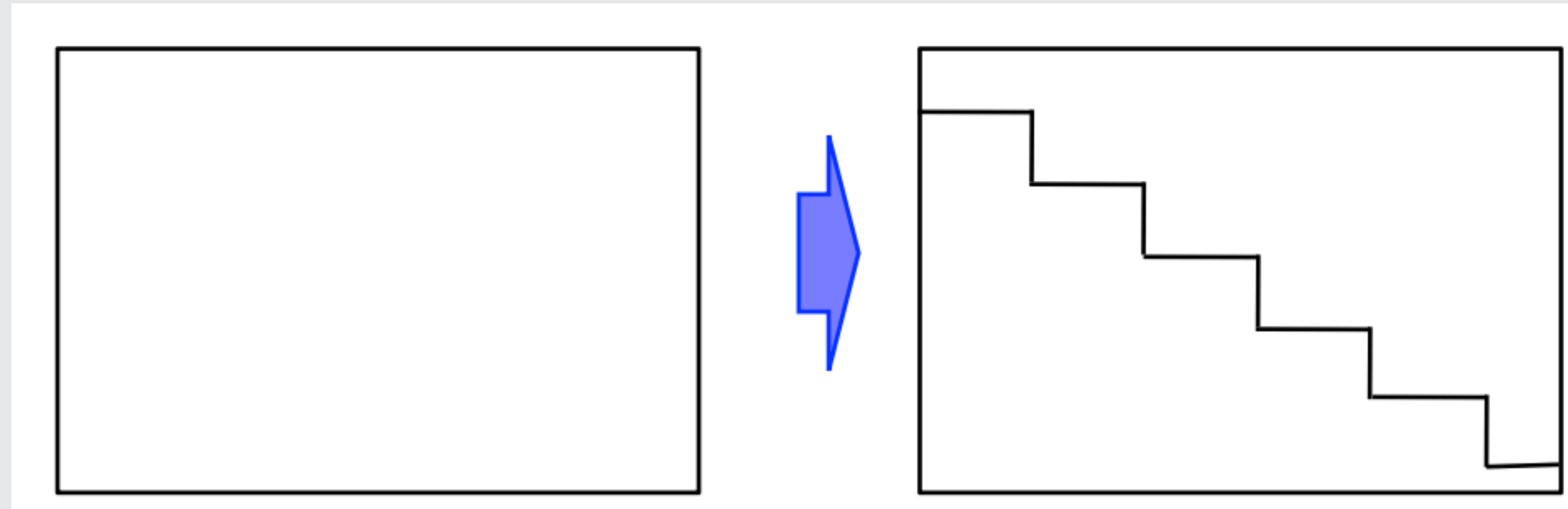
Esercizio :

Uso del comando `for` in struttura bi-dimensionale (A3)

Scrivere una funzione che traccia una "scaletta" all'interno di un'immagine bianca.

Sulle slide viene riportato anche che bisogna utilizzare i seguenti parametri:

- Punto di inizio
- Dimensione "scalini"
- Colore delle linee



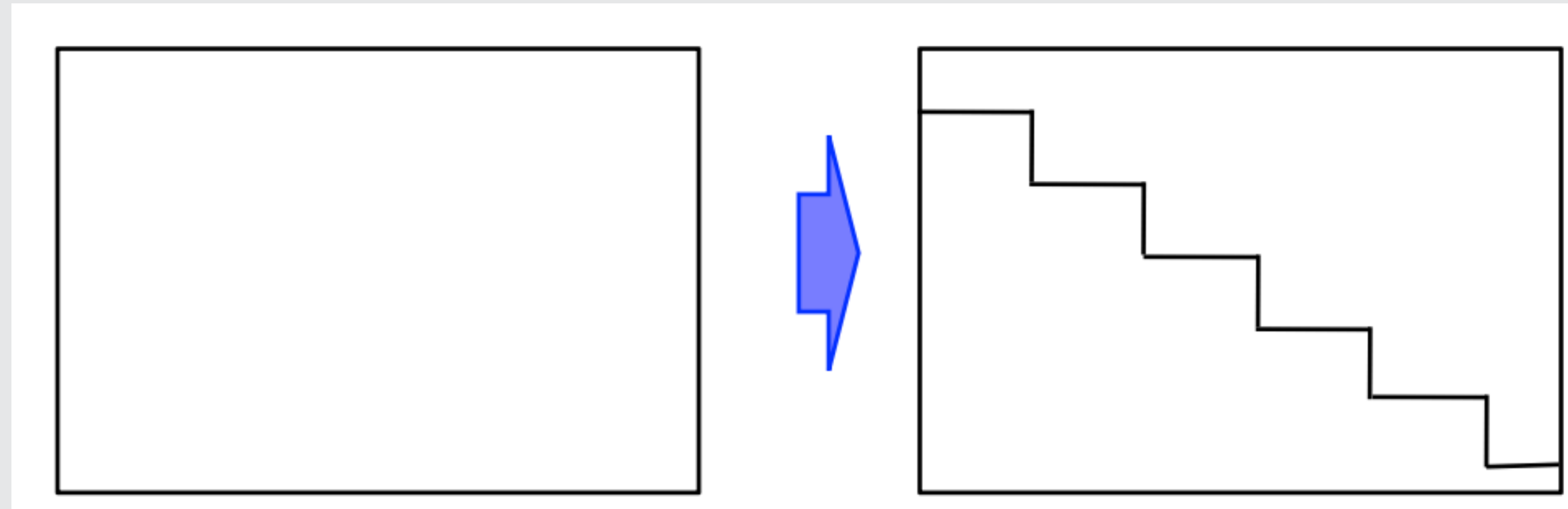
Esercizio :

Uso del comando `for` in struttura bi-dimensionale (A3)

Scrivere una funzione che traccia una "scaletta" all'interno di un'immagine bianca.

Sulle slide viene riportato anche che bisogna utilizzare i seguenti parametri:

- Punto di inizio
- Dimensione "scalini"
- Colore delle linee



**Attenzione a non uscire
dal bordo!**

Esercizio :

Uso del comando for in struttura bi-dimensionale (A3)

Creiamo un sotto problema:

- Assumendo che xStart e yStart abbiano valori sempre maggiori di 0. Siamo in grado di aggiungere un gradino all'interno di un immagine?
- Assumiamo che i parametri siano li stessi del problema principale



Esercizio :

Uso del comando for in struttura bi-dimensionale (A3)

Soluzione del sottoproblema

```
1 def disegnaGradino(pict,xStart,yStart,width,height,col):
2     #@Param pict:Picture
3     #@Param xStart:integer
4     #@Param yStart:integer
5     #@Param width:integer
6     #@Param height:integer
7     #@Param col:Color
8     pictWidth=getWidth(pict)
9     pictHeight=getHeight(pict)
10    xUpperBound = min(xStart+width,pictWidth)
11    print xStart+width, pictWidth, xUpperBound
12    if yStart < pictHeight:
13        for x in range (xStart,xUpperBound): #qui il meno uno e' implicito
14            p=getPixel(pict,x,yStart)
15            setColor(p,col)
16    yUpperBound = min(yStart+height,pictHeight)
17    print yStart+height, pictHeight, yUpperBound
18    if xUpperBound < pictWidth:
19        for y in range (yStart,yUpperBound):
20            p=getPixel(pict,xUpperBound,y) #qui lo devo esplicitare
21            setColor(p,col)
```

Esercizio :

Uso del comando for in struttura bi-dimensionale (A3)

Soluzione del sottoproblema

```
1 def disegnaGradino(pict,xStart,yStart,width,height,col):
2     #@Param pict:Picture
3     #@Param xStart:integer
4     #@Param yStart:integer
5     #@Param width:integer
6     #@Param height:integer
7     #@Param col:Color
8     pictWidth=getWidth(pict)
9     pictHeight=getHeight(pict)
10    xUpperBound = min(xStart+width,pictWidth)
11    print xStart+width, pictWidth, xUpperBound
12    if yStart < pictHeight:
13        for x in range (xStart,xUpperBound): #qui il meno uno e' implicito
14            p=getPixel(pict,x,yStart)
15            setColor(p,col)
16    yUpperBound = min(yStart+height,pictHeight)
17    print yStart+height, pictHeight, yUpperBound
18    if xUpperBound < pictWidth:
19        for y in range (yStart,yUpperBound):
20            p=getPixel(pict,xUpperBound,y) #qui lo devo esplicitare
21            setColor(p,col)
```

ciclo che permette di disegnare una linea orizzontale

ciclo che permette di disegnare una linea orizzontale

Esercizio :

Uso del comando for in struttura bi-dimensionale (A3)

Soluzione del sottoproblema

```
1 def disegnaGradino(pict,xStart,yStart,width,height,col):
2     #@Param pict:Picture
3     #@Param xStart:integer
4     #@Param yStart:integer
5     #@Param width:integer
6     #@Param height:integer
7     #@Param col:Color
8     pictWidth=getWidth(pict)
9     pictHeight=getHeight(pict)
10    xUpperBound = min(xStart+width,pictWidth)
11    print xStart+width, pictWidth, xUpperBound
12    if yStart < pictHeight:
13        for x in range (xStart,xUpperBound): #qui il meno uno e' implicito
14            p=getPixel(pict,x,yStart)
15            setColor(p,col)
16    yUpperBound = min(yStart+height,pictHeight)
17    print yStart+height, pictHeight, yUpperBound
18    if xUpperBound < pictWidth:
19        for y in range (yStart,yUpperBound):
20            p=getPixel(pict,xUpperBound,y) #qui lo devo esplicitare
21            setColor(p,col)
```

Controllo se entra un
ulteriore riga

Controllo che entra
un'ulteriore colonna

Esercizio :

Uso del comando for in struttura bi-dimensionale (A3)

Soluzione del sottoproblema

```
1 def disegnaGradino(pict,xStart,yStart,width,height,col):
2     #@Param pict:Picture
3     #@Param xStart:integer
4     #@Param yStart:integer
5     #@Param width:integer
6     #@Param height:integer
7     #@Param col:Color
8     pictWidth=getWidth(pict)
9     pictHeight=getHeight(pict)
10    xUpperBound = min(xStart+width,pictWidth)
11    print xStart+width, pictWidth, xUpperBound
12    if yStart < pictHeight:
13        for x in range (xStart,xUpperBound): #qui il meno uno e' implicito
14            p=getPixel(pict,x,yStart)
15            setColor(p,col)
16    yUpperBound = min(yStart+height,pictHeight)
17    print yStart+height, pictHeight, yUpperBound
18    if xUpperBound < pictWidth:
19        for y in range (yStart,yUpperBound):
20            p=getPixel(pict,xUpperBound,y) #qui lo devo esplicitare
21            setColor(p,col)
```

Limite superiore della
lunghezza della riga
orizzontale

Limiti superiore della
della riga verticale

Esercizio :

Uso del comando for in struttura bi-dimensionale (A3)

Soluzione del problema principale:

```
23 def disegnaScala(pict,xStart,yStart,width,height,col):
24     #@Param pict:Picture
25     #@Param xStart:integer
26     #@Param yStart:integer
27     #@Param width:integer
28     #@Param height:integer
29     #@Param col:Color
30     pictWidth=getWidth(pict)
31     pictHeight=getHeight(pict)
32     if xStart <0 or xStart >= pictWidth:
33         return
34     if yStart <0 or yStart >= pictHeight:
35         return
36     numGradini = min((pictWidth-xStart)/width,(pictHeight-yStart)/height)+1
37     x = xStart
38     y = yStart
39     for i in range(0,numGradini):
40         disegnaGradino(pict,x,y,width,height,col)
41         x = x + width
42         y = y + height
```

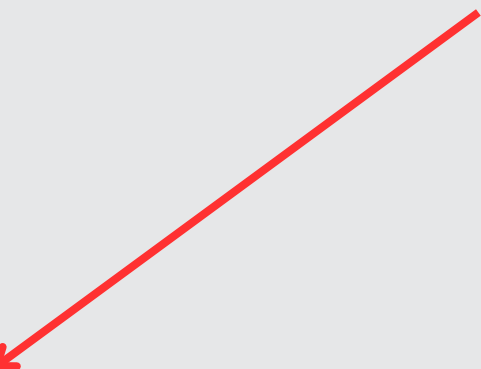
Esercizio :

Uso del comando for in struttura bi-dimensionale (A3)

Soluzione del problema principale:

```
23 def disegnaScala(pict,xStart,yStart,width,height,col):
24     #@Param pict:Picture
25     #@Param xStart:integer
26     #@Param yStart:integer
27     #@Param width:integer
28     #@Param height:integer
29     #@Param col:Color
30     pictWidth=getWidth(pict)
31     pictHeight=getHeight(pict)
32     if xStart <0 or xStart >= pictWidth:
33         return
34     if yStart <0 or yStart >= pictHeight:
35         return
36     numGradini = min((pictWidth-xStart)/width,(pictHeight-yStart)/height)+1
37     x = xStart
38     y = yStart
39     for i in range(0,numGradini):
40         disegnaGradino(pict,x,y,width,height,col)
41         x = x + width
42         y = y + height
```

calcolo il numero di gradini



Esercizio :

Uso del comando for in struttura bi-dimensionale (A3)

Soluzione del problema principale:

```
23 def disegnaScala(pict,xStart,yStart,width,height,col):
24     #@Param pict:Picture
25     #@Param xStart:integer
26     #@Param yStart:integer
27     #@Param width:integer
28     #@Param height:integer
29     #@Param col:Color
30     pictWidth=getWidth(pict)
31     pictHeight=getHeight(pict)
32     if xStart <0 or xStart >= pictWidth:
33         return
34     if yStart <0 or yStart >= pictHeight:
35         return
36     numGradini = min((pictWidth-xStart)/width,(pictHeight-yStart)/height)+1
37     x = xStart
38     y = yStart
39     for i in range(0,numGradini):
40         disegnaGradino(pict,x,y,width,height,col)
41         x = x + width
42         y = y + height
```

Per ogni gradino eseguo la funzione
disengaGradino
Cambiando il punto di inizio
gradino